

Classical Machine Learning

**A Napkin-style companion for campus placements:
derivations, hand numerics, traps, and code that checks itself**

Philospher

Built October 6, 2026

Preface

[STUB: preface]

Contents

I	Foundations	9
1	The Setup	11
1.1	The zoo of learning problems	11
1.2	Loss, risk and empirical risk	12
1.3	Train, validation, test	14
1.4	Over- and underfitting, the generalisation gap	14
1.5	The i.i.d. assumption and when it breaks	16
1.6	A few problems to think about	16
2	Maths at Your Fingertips	19
2.1	Linear algebra	19
2.2	Calculus and matrix calculus	20
2.3	Probability	21
2.4	Information theory in five lines	23
2.5	Maximum likelihood and MAP	23
2.6	Convexity	24
2.7	Lagrange multipliers and KKT	26
2.8	A few problems to think about	27
II	Supervised Learning	29
3	Linear Regression	33
3.1	Least squares and the normal equations	33
3.2	The projection and MLE views	34
3.3	Five points by hand	35
3.4	Gradient descent, traced	36
3.5	R^2 and adjusted R^2	37
3.6	Assumptions, diagnostics, multicollinearity	38
3.7	Polynomial features and robust losses	39
3.8	A few problems to think about	40
4	Bias, Variance and Regularisation	41
4.1	The decomposition	41
4.2	Ridge	41
4.3	Lasso and elastic net	41
4.4	The augmented error	41
4.5	Early stopping and double descent	41
5	Logistic Regression and GLMs	43
5.1	From odds to the sigmoid	43
5.2	Log-loss from MLE	43
5.3	Newton and IRLS	43
5.4	Softmax regression and multiclass	43
5.5	Imbalance and thresholds	43
5.6	The perceptron and XOR	43

6	Generative Classifiers	45
6.1	Naive Bayes	45
6.2	Laplace smoothing	45
6.3	LDA and QDA	45
7	k-Nearest Neighbours and Distances	47
7.1	The algorithm	47
7.2	Distances	47
7.3	Choosing k	47
7.4	The curse of dimensionality	47
7.5	KD-trees and approximate search	47
8	Support Vector Machines	49
8.1	Margins	49
8.2	Hard and soft margin	49
8.3	Duality	49
8.4	Kernels	49
8.5	SVR and SMO	49
8.6	Convex surrogate losses	49
9	Decision Trees	51
9.1	Impurity	51
9.2	A split by hand	51
9.3	Growing a tree	51
9.4	Regression trees	51
9.5	Pruning	51
9.6	What trees are good for	51
10	Ensembles	53
10.1	The bootstrap and 63.2 percent	53
10.2	Bagging and random forests	53
10.3	Feature importance and its traps	53
10.4	AdaBoost	53
10.5	Gradient boosting	53
10.6	XGBoost	53
10.7	LightGBM and CatBoost	53
10.8	Stacking and the bias-variance story	53
III	Learning Without (Enough) Labels	55
11	Unsupervised Learning	57
11.1	k-means	57
11.2	Hierarchical clustering	57
11.3	DBSCAN	57
11.4	Gaussian mixtures and EM	57
11.5	PCA	57
11.6	ICA, t-SNE and UMAP	57
11.7	Anomaly detection	57
11.8	Association rules	57

12	Semi- and Self-Supervised Learning	59
12.1	Assumptions	59
12.2	Self-training	59
12.3	Co-training and label propagation	59
12.4	Consistency regularisation	59
12.5	When unlabelled data hurts	59
12.6	Contrastive learning in one page	59
IV	Doing It Properly	61
13	Evaluation and Model Selection	63
13.1	The confusion matrix	63
13.2	ROC and PR curves	63
13.3	Regression and probabilistic metrics	63
13.4	Cross-validation	63
13.5	Leakage	63
13.6	AIC, BIC and hyper-parameter search	63
13.7	Calibration	63
14	Statistics Interviewers Ask	65
14.1	Distributions	65
14.2	Estimators and confidence intervals	65
14.3	Hypothesis tests	65
14.4	Chi-square and ANOVA	65
14.5	Multiple testing	65
14.6	Correlation, causation and Simpson	65
15	Data Preparation and Feature Engineering	67
15.1	Scaling	67
15.2	Encoding categoricals	67
15.3	Missing values and outliers	67
15.4	Feature selection	67
15.5	Imbalance	67
15.6	Text features and TF-IDF	67
15.7	Time series	67
15.8	Recommenders	67
16	Optimisation for ML	69
16.1	Gradient descent on a quadratic	69
16.2	Momentum	69
16.3	Newton and quasi-Newton	69
16.4	Coordinate descent	69
16.5	Duality	69
V	Interview Mode	71
17	Implement It	73
17.1	Linear models	73
17.2	Neighbours and Bayes	73

17.3	Trees and forests	73
17.4	Boosting	73
17.5	Clustering and PCA	73
17.6	SVM	73
18	Rapid-fire	75
18.1	Rapid-fire answers	75
19	Problems	77
19.1	Problems	77
VI	Backmatter	79
A	Hints to problems	81
B	Solutions to problems	83

I

Foundations

Part I: Contents

1	The Setup	11
1.1	The zoo of learning problems	11
1.2	Loss, risk and empirical risk	12
1.3	Train, validation, test	14
1.4	Over- and underfitting, the generalisation gap	14
1.5	The i.i.d. assumption and when it breaks	16
1.6	A few problems to think about	16
2	Maths at Your Fingertips	19
2.1	Linear algebra	19
2.2	Calculus and matrix calculus	20
2.3	Probability	21
2.4	Information theory in five lines	23
2.5	Maximum likelihood and MAP	23
2.6	Convexity	24
2.7	Lagrange multipliers and KKT	26
2.8	A few problems to think about	27

1 The Setup

Every interview question in this book is secretly about one sentence: *we choose a function using a finite sample, and we are paid for how it does on data we have not seen*. This chapter fixes the vocabulary for that sentence. It is short on algorithms and long on the distinctions that one-line MCQs love to blur: loss versus risk, validation versus test, training error versus generalisation gap.

§1.1 The zoo of learning problems

Prototypical example for this section: A bank has two million past loan applications, each with a “defaulted / repaid” label, and wants a score for tomorrow’s applicant.

The type of problem is decided by *what supervision you have at training time*, not by the algorithm you later pick.

Definition 1.1.1 (The five regimes). • **Supervised learning**: each training example is a pair $(\mathbf{x}_i, y_i)$, and we learn f with $f(\mathbf{x}) \approx y$. If y is real-valued it is **regression**; if y lies in a finite set it is **classification** (binary or multiclass; **multilabel** if several labels can be on at once).

- **Unsupervised learning**: only $\mathbf{x}_i$. We look for structure: clusters, low-dimensional coordinates, densities, anomalies.
- **Semi-supervised learning**: a few labelled pairs plus many unlabelled $\mathbf{x}$ ’s, and the goal is still a supervised predictor (**Chapter 12**).
- **Self-supervised learning**: labels are *manufactured from the input itself* (mask a word and predict it; crop two views of an image and ask whether they match). It is trained exactly like supervised learning, but nobody paid an annotator.
- **Reinforcement learning**: an agent acts, the environment returns states and a scalar **reward**, and the goal is a policy that maximises long-run reward. There is no correct action given per example, only delayed, noisy feedback on the actions you happened to take.

A useful everyday picture: supervised learning is a student with an answer key, unsupervised is a librarian sorting books with no catalogue, semi-supervised is the librarian who was shown how fifty books were shelved, self-supervised is a student who covers half a page and tries to recall it, and reinforcement learning is learning to cycle — nobody tells you the correct handlebar angle, you just fall less over time.

Task	Regime	Output
Price of a flat from area, floor, locality	supervised	regression
Is this transaction fraud?	supervised	binary classification
Which of 10 digits is in the image?	supervised	multiclass
Which genres does this film belong to?	supervised	multilabel
Segment customers by purchase behaviour	unsupervised	clustering
Flag unusual server logs (no labels)	unsupervised	anomaly score
Pre-train a language model by predicting masked words	self-supervised	classification over vocabulary
Learn to place bids in an ad auction from revenue	reinforcement	policy

Remark 1.1.2 — Trap. “Predicting the number of goals in a match (0, 1, 2, ...) is classification because the output is discrete.” Counts are ordered and arithmetic on them is meaningful, so the usual answer is *regression* (often Poisson regression, [Chapter 5](#)). Discreteness alone does not make a problem classification; *unordered* categories do. Similarly “customer segmentation” is clustering, not classification, unless someone hands you segment labels.

Remark 1.1.3 — Trap. Self-supervised $\neq$ unsupervised in an MCQ. Both use unlabelled data, but self-supervised learning optimises an ordinary supervised loss on a *pretext task*, and its output is usually a representation reused downstream. If an option says “BERT pre-training is unsupervised”, many keys accept it; the precise term is self-supervised.

§1.1.i Parametric versus non-parametric; discriminative versus generative

A **parametric** model has a fixed number of parameters regardless of n (linear and logistic regression). A **non-parametric** model grows with the data (k -nearest neighbours stores all of it; a fully grown tree can have up to n leaves). A **discriminative** model learns $p(y | \mathbf{x})$ or a decision rule directly (logistic regression, SVM); a **generative** model learns $p(\mathbf{x} | y)$ and $p(y)$ and applies Bayes’ rule (naive Bayes, LDA, [Chapter 6](#)).

The regime is set by the supervision available at training time; the output type (real, class, set of classes) then decides regression versus classification.

§1.2 Loss, risk and empirical risk

Prototypical example for this section: Predict $\hat{y} = 2x + 1$ on five points and average the squared errors.

Definition 1.2.1. A **loss** $\ell(\hat{y}, y) \geq 0$ scores one prediction. Common ones: squared loss $(\hat{y} - y)^2$, absolute loss $|\hat{y} - y|$, **0-1 loss** $\mathbb{I}[\hat{y} \neq y]$, log-loss $-\log \hat{p}_y$. Assume data come from an unknown distribution $\mathcal{D}$ over $(\mathbf{x}, y)$. The **risk** (true or expected risk, generalisation error) of f is

$$R(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\ell(f(\mathbf{x}), y)],$$

and the **empirical risk** on a sample $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ is

$$\hat{R}_S(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i), y_i).$$

Empirical risk minimisation (ERM) picks $\hat{f} = \arg \min_{f \in \mathcal{H}} \hat{R}_S(f)$ over a **hypothesis class** $\mathcal{H}$.

Loss is per example, risk is an expectation over the population, empirical risk is an average over your sample. MCQs swap these words freely; you should not.

Example 1.2.2 (Empirical risk by hand)

Points $x = 0, 1, 2, 3, 4$ with $y = 1.2, 2.9, 5.1, 7.2, 8.7$ and the predictor $f(x) = 2x + 1$. Predictions are 1, 3, 5, 7, 9, so the residuals $y - f(x)$ are

$$+0.2 \quad -0.1 \quad +0.1 \quad +0.2 \quad -0.3 \quad .$$

Mean squared error = 0.038; mean absolute error = 0.180. For a 0-1 example, the rule “predict 1 iff $x > 2$ ” on $x = (0.5, 1.5, 2.2, 2.8, 3.5, 4.1)$, $y = (0, 0, 1, 0, 1, 1)$ makes 1 mistake (the point $x = 2.8$), so the empirical 0-1 risk is $1/6 = 0.1667$.

§1.2.i The best you can possibly do

Even a perfect learner cannot beat the noise in y given $\mathbf{x}$.

Definition 1.2.3. The **Bayes predictor** minimises the risk over *all* functions. Under squared loss it is $f^*(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$; under 0-1 loss it is $f^*(\mathbf{x}) = \arg \max_c \mathbb{P}(y = c \mid \mathbf{x})$. Its risk is the **Bayes risk** (**Bayes error** for 0-1 loss), the irreducible error.

Why the conditional mean is optimal for squared loss. For fixed $\mathbf{x}$ write $m = \mathbb{E}[y \mid \mathbf{x}]$. Then $\mathbb{E}[(c - y)^2 \mid \mathbf{x}] = (c - m)^2 + \text{Var}(y \mid \mathbf{x})$, minimised at $c = m$; the leftover $\text{Var}(y \mid \mathbf{x})$ is the Bayes risk at $\mathbf{x}$. $\square$

Example 1.2.4 (Bayes error of two Gaussians)

Class 0 is $\mathcal{N}(-1, 1)$, class 1 is $\mathcal{N}(+1, 1)$, equal priors. By symmetry the Bayes rule thresholds at 0, and its error is $\Phi(-1) = 0.1587$; a simulation with 200 000 draws gives 0.1599. No classifier, however deep, does better on this distribution. With priors 0.8/0.2 the optimal threshold moves to $\frac{1}{2} \ln(0.8/0.2) = 0.6931$ (towards the rare class) and the error drops to 0.1121; keeping the threshold at 0 would leave it at 0.1587.

Remark 1.2.5 — Trap. “Bayes error is the error of naive Bayes.” No: Bayes error is a property of the *distribution*, the floor for every classifier. Naive Bayes is one particular (usually sub-optimal) classifier.

§1.2.ii No free lunch, inductive bias

Averaged over *all* possible labelling functions, every learner has the same off-training-set error. Learning works only because real problems are not uniformly random: we assume

smoothness, linearity, sparsity, locality, invariance. These assumptions are the model's **inductive bias**, and choosing $\mathcal{H}$ is choosing one. “Which algorithm is best?” has no answer without a data distribution.

§1.3 Train, validation, test

Prototypical example for this section: Fifty models compared on the same validation set; the winner's score is too good to be true.

Definition 1.3.1. The **training set** fits parameters. The **validation set** (or cross-validation, [Chapter 13](#)) chooses among models and **hyper-parameters** (anything not learnt by the fitting procedure itself: degree, k , λ , tree depth). The **test set** is touched *once*, at the end, to report an unbiased estimate of risk.

Why three sets and not two? Every time you pick the best of several candidates by a score, the winning score is biased upwards: you selected partly on luck. The test set must not have taken part in any choice.

Example 1.3.2 (Selection optimism)

Take 50 “models” that are pure coin flips and score each on a balanced validation set of 100 points. Every model has true accuracy 0.5, yet over 2000 repetitions the *best* validation accuracy averages 0.612 (exact value from the distribution of the maximum of 50 binomials: 0.612). On fresh data the winner scores 0.500. A twelve-point “improvement” produced by selection alone.

Remark 1.3.3 — Typical splits: 60/20/20 or 80/10/10 for moderate data; with millions of rows a 1% test set is already tens of thousands of examples. With little data use k -fold cross-validation for selection and keep a held-out test set if you can afford it; nested CV otherwise ([Chapter 13](#)).

Remark 1.3.4 — **Trap.** Tuning on the test set (“I tried a few depths and kept the one with best test accuracy”) turns the test set into a validation set. The reported number is then optimistic, exactly as in the coin-flip example. Interviewers probe this with “how many times did you look at the test set?”

§1.4 Over- and underfitting, the generalisation gap

Prototypical example for this section: Fit polynomials of degree 0 to 12 to fifteen noisy samples of $\sin 2\pi x$.

Definition 1.4.1. The **generalisation gap** of $\hat{f}$ is $R(\hat{f}) - \hat{R}_S(\hat{f})$: test (or validation) error minus training error, or equivalently training accuracy minus test accuracy. A model **underfits** when even its training error is high (too little capacity or too much regularisation), and **overfits** when training error is low but the gap is large (it has memorised noise specific to S).

Example 1.4.2 (OA one-liner)

Training accuracy 0.98, test accuracy 0.81. The generalisation gap is 0.17 (17 percentage points): overfitting. Fixes, in order of what interviewers want to hear: more data, regularisation, simpler model / fewer features, early stopping, ensembling (bagging), data augmentation.

Example 1.4.3 (The degree sweep)

Fifteen training points, $y = \sin(2\pi x) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 0.3^2)$ (so the Bayes risk is 0.09). Median MSE over 200 repetitions:

degree	train MSE	test MSE	gap
0	0.558	0.611	0.053
1	0.238	0.323	0.085
3	0.065	0.124	0.059
9	0.037	0.777	0.740

Degree 0 and 1 underfit (both errors high, gap small); degree 3 is best on test; degree 9 has the lowest training error and a huge gap. The *mean* test MSE at degree 9 is 3768, because a few unlucky fits explode between and beyond the training points — variance in its purest form.

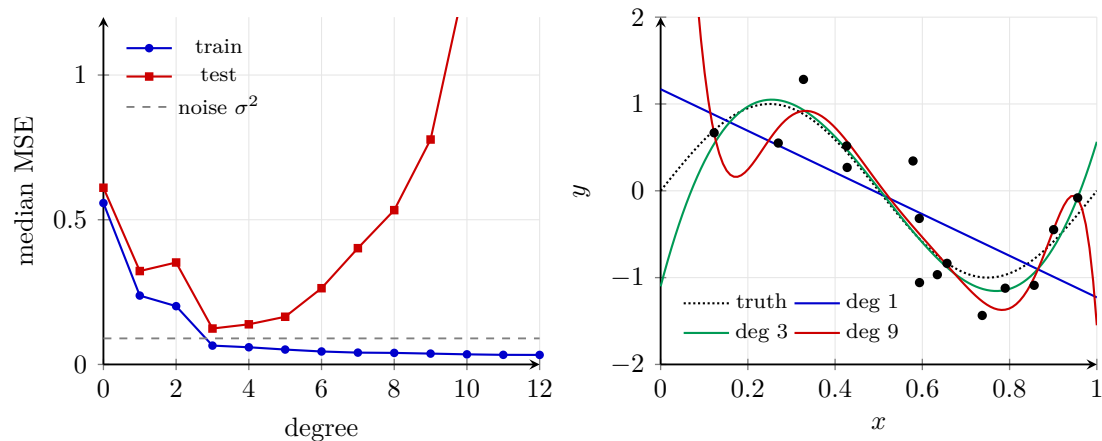


Figure 1.1: Left: training error falls monotonically with capacity; test error is U-shaped. Right: degree 1 underfits, degree 3 is about right, degree 9 threads every noisy point.

Remark 1.4.4 — Trap. “A small generalisation gap means a good model.” A constant predictor has a tiny gap and is useless. You need *both* low training error and a small gap. Conversely a modest gap with the best validation score is fine; deep nets routinely have training error near zero and are still the best model available.

Remark 1.4.5 — Trap. “Training error keeps decreasing, so we are still improving.” Training error is non-increasing in capacity for nested model classes almost by definition; it says nothing about generalisation. Watch validation error.

Question 1.4.6. If you add an irrelevant random feature to a least-squares model, can its training MSE go up? (Answer: no — the old solution is still available with weight 0 on the new feature.)

§1.5 The i.i.d. assumption and when it breaks

Prototypical example for this section: A model trained on 2022 transactions is scored on 2024 transactions after a payment app changes its UI.

Everything above assumes the training points and future points are drawn **independently and identically distributed (i.i.d.)** from the same $\mathcal{D}$. The two halves fail differently.

- **Not independent.** Several rows per patient, per user, per session; consecutive time steps. A random row-level split puts the same patient in train and test, and the model scores well by recognising the patient. Fix: split by group (GroupKFold) or by time.
- **Not identically distributed (distribution shift).** **Covariate shift:** $p(\mathbf{x})$ changes, $p(y | \mathbf{x})$ does not (a new city with different house sizes). **Label shift:** $p(y)$ changes, $p(\mathbf{x} | y)$ does not (fraud rate doubles during a festival sale). **Concept drift:** $p(y | \mathbf{x})$ itself changes (what counts as spam evolves).

Remark 1.5.1 — Trap. Shuffling a time series before a train/test split leaks the future into the past: the model sees Tuesday when predicting Monday. Use forward-chaining splits (Chapter 13).

Training error measures fit; the gap measures luck you mistook for signal; the i.i.d. assumption is what lets either number say anything about tomorrow.

§1.6 A few problems to think about

Problem 1A (Name the regime). Classify each as supervised (regression/classification), unsupervised, self-supervised or RL: (a) predicting tomorrow’s maximum temperature; (b) grouping news articles into topics with no labels; (c) predicting the next frame of a video from previous frames; (d) learning to play Ludo from win/loss outcomes; (e) assigning one of five star ratings to a review.

hint · solution

Problem 1B (Empirical risk). A predictor outputs $\hat{y} = (3, 5, 2, 8)$ for targets $y = (2, 5, 4, 7)$. Compute the mean squared error and the mean absolute error. Which of the two is more affected if the last target were 17 instead of 7?

hint · solution

Problem 1C (Gap arithmetic). Model A: train accuracy 0.99, validation 0.84. Model B: train 0.88, validation 0.86. Model C: train 0.71, validation 0.70. Which overfits, which underfits, which would you ship?

[hint](#) · [solution](#)

Problem 1D (Bayes rule with priors). Class 0 is $\mathcal{N}(0, 1)$ and class 1 is $\mathcal{N}(2, 1)$, with $\mathbb{P}(y = 1) = \pi$. Show that the Bayes classifier predicts 1 iff $x > 1 + \frac{1}{2} \ln \frac{1-\pi}{\pi}$. What happens as $\pi \rightarrow 0$?

[hint](#) · [solution](#)

Problem 1E (Leaky split). A hospital dataset has 10 X-rays each for 500 patients; the label is a diagnosis per patient. A random 80/20 split of the 5000 images gives 97% test accuracy, but the deployed model gets 78%. Explain, and propose a correct validation scheme.

[hint](#) · [solution](#)

Problem 1F (Selection bias, quantified). You evaluate m useless classifiers (each predicts with a fair coin) on a balanced validation set of size n . Using the normal approximation, estimate the expected best validation accuracy for $m = 50$, $n = 100$. (Use $\mathbb{E}[\max \text{ of } 50 \text{ standard normals}] \approx 2.25$.)

[hint](#) · [solution](#)

2 Maths at Your Fingertips

This chapter is the toolkit. Nothing here is decoration: every later derivation (normal equations, ridge, logistic gradients, the SVM dual, PCA, EM) is one of these facts applied once. Read it quickly, then come back when a later chapter says “by the identity in Chapter 2”.

§2.1 Linear algebra

Prototypical example for this section: The hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$, which projects $\mathbf{y}$ onto the column space of $\mathbf{X}$.

§2.1.i Vectors, norms, inner products

For $\mathbf{x} \in \mathbb{R}^d$, the ℓ_p **norm** is $\|\mathbf{x}\|_p = (\sum_i |x_i|^p)^{1/p}$. The three you need: $\|\mathbf{x}\|_1 = \sum |x_i|$ (lasso), $\|\mathbf{x}\|_2 = \sqrt{\sum x_i^2}$ (ridge, Euclidean distance), $\|\mathbf{x}\|_\infty = \max |x_i|$. For $\mathbf{x} = (3, -4, 12)$: $\|\mathbf{x}\|_1 = 19$, $\|\mathbf{x}\|_2 = 13$, $\|\mathbf{x}\|_\infty = 12$. Always $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1$.

The inner product $\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$; **cosine similarity** is $\cos \theta$, which ignores length — why it is the default for comparing TF-IDF vectors and embeddings, where document length is a nuisance (Chapter 15).

§2.1.ii Rank, inverses, conditioning

The **rank** of $\mathbf{A}$ is the dimension of its column space. $\mathbf{A} \in \mathbb{R}^{d \times d}$ is invertible iff rank d iff $\det \mathbf{A} \neq 0$ iff no eigenvalue is 0. In regression, $\mathbf{X}^\top \mathbf{X}$ is singular exactly when a feature is a linear combination of others (perfect multicollinearity, Chapter 3); when some $n < d$ it is *always* singular. The **condition number** $\kappa(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$ measures how much relative error in $\mathbf{b}$ can be amplified in the solution of $\mathbf{A}\mathbf{x} = \mathbf{b}$; for $\text{diag}(1, 10^{-4})$ it is 10000. Large κ also makes gradient descent zig-zag (Chapter 16).

§2.1.iii Eigenvalues and the spectral theorem

$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ with $\mathbf{v} \neq \mathbf{0}$. A real symmetric matrix has real eigenvalues and an orthonormal eigenbasis: $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ (**spectral theorem**). Useful facts: $\text{Tr } \mathbf{A} = \sum \lambda_i$, $\det \mathbf{A} = \prod \lambda_i$, and eigenvalues of $\mathbf{A}^k$ are λ_i^k .

Example 2.1.1 (A rank-one matrix)

$\mathbf{S} = \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix}$ has trace 5 and determinant 0.000, so its eigenvalues are 5.000 and 0.000 and its rank is 1 (it is $(2, 1)^\top (2, 1)$). Trace and determinant give a 2×2 's eigenvalues in ten seconds: they are the roots of $\lambda^2 - (\text{Tr})\lambda + \det$.

§2.1.iv Positive semi-definite matrices

Definition 2.1.2. Symmetric $\mathbf{A}$ is **positive semi-definite** (PSD, $\mathbf{A} \succeq 0$) if $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ for all $\mathbf{x}$, and **positive definite** (PD) if > 0 for all $\mathbf{x} \neq \mathbf{0}$. Equivalently: all eigenvalues ≥ 0 (resp. > 0).

Every covariance matrix and every Gram matrix $\mathbf{X}^\top \mathbf{X}$ is PSD, because $\mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = \|\mathbf{X} \mathbf{v}\|^2 \geq 0$. Adding $\lambda \mathbf{I}$ with $\lambda > 0$ makes a PSD matrix PD — the whole reason ridge regression always has a unique solution. Kernel matrices must be PSD (Mercer, Chapter 8). $\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$ is PD (eigenvalues 1.000, 3.000); a diagonal-dominance glance tells you so.

§2.1.v Singular value decomposition

Every $\mathbf{A} \in \mathbb{R}^{n \times d}$ factors as $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$ with orthonormal columns in $\mathbf{U}, \mathbf{V}$ and non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \dots$ on the diagonal of $\mathbf{\Sigma}$. The σ_i^2 are the eigenvalues of $\mathbf{A}^\top \mathbf{A}$ (and of $\mathbf{A} \mathbf{A}^\top$), $\mathbf{V}$ holds the eigenvectors of $\mathbf{A}^\top \mathbf{A}$.

Theorem 2.1.3 (Eckart–Young)

The best rank- k approximation of $\mathbf{A}$ in Frobenius (or spectral) norm is the truncated SVD $\mathbf{A}_k = \sum_{i \leq k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, with error $\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sum_{i > k} \sigma_i^2}$.

Example 2.1.4 (SVD by eigenvalues)

$\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \\ 1 & 1 \end{pmatrix}$ gives $\mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 11 & 7 \\ 7 & 11 \end{pmatrix}$ with eigenvalues 11 ± 7 , i.e. 18.000 and 4.000, so $\sigma_1 = \sqrt{18} = 4.2426$ and $\sigma_2 = 2.0000$. The best rank-one approximation misses by exactly $\sigma_2 = 2.0000$ in Frobenius norm. PCA (Chapter 11) is this theorem applied to a centred data matrix.

§2.1.vi Projections

The orthogonal projection onto the column space of a full-column-rank $\mathbf{X}$ is $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$. It is symmetric and **idempotent** ($\mathbf{H}^2 = \mathbf{H}$: projecting twice changes nothing), its eigenvalues are 0 or 1, and $\text{Tr } \mathbf{H} = \text{rank } \mathbf{X}$.

Example 2.1.5 (Projecting onto a line)

With $\mathbf{X} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}$ (intercept and $x = 0, 1, 2$) and $\mathbf{y} = (1, 2, 2)$, the projection is $\hat{\mathbf{y}} = \mathbf{H} \mathbf{y} = (1.167, 1.667, 2.167)$ — the fitted values of the least-squares line — and $\text{Tr } \mathbf{H} = 2.000$, the number of parameters. The residual $\mathbf{y} - \hat{\mathbf{y}}$ is orthogonal to every column of $\mathbf{X}$.

Symmetric $\Rightarrow$ orthogonal eigenvectors; Gram and covariance matrices are PSD; adding $\lambda \mathbf{I}$ makes them invertible; least squares is a projection.

§2.2 Calculus and matrix calculus

Prototypical example for this section: $\nabla_w \|\mathbf{X} \mathbf{w} - \mathbf{y}\|^2 = 2 \mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y})$, the gradient behind every linear model.

§2.2.i Gradient, Jacobian, Hessian

For $f : \mathbb{R}^d \rightarrow \mathbb{R}$, the **gradient** $\nabla f \in \mathbb{R}^d$ collects partial derivatives and points in the direction of steepest ascent; $-\nabla f$ is the descent direction. For $\mathbf{f} : \mathbb{R}^d \rightarrow \mathbb{R}^m$ the **Jacobian** $\mathbf{J} \in \mathbb{R}^{m \times d}$ has $J_{ij} = \partial f_i / \partial x_j$; the chain rule is a product of Jacobians, which is all backpropagation is (see *Deep Learning*). The **Hessian** $\nabla^2 f$ is the $d \times d$ matrix of second derivatives; it is symmetric for smooth f .

At a stationary point ($\nabla f = \mathbf{0}$): Hessian PD $\Rightarrow$ local minimum, ND $\Rightarrow$ local maximum, eigenvalues of both signs $\Rightarrow$ **saddle point**. For $f = x^2 + 3xy + y^2$ the Hessian $\begin{pmatrix} 2 & 3 \\ 3 & 2 \end{pmatrix}$ has eigenvalues 5 and -1 , so the origin is a saddle, although the function is a “bowl” along both axes.

§2.2.ii The identities you will use

With $\mathbf{A}$ a constant matrix and $\mathbf{b}$ a constant vector (“numerator layout” gradients as column vectors):

$$\begin{aligned}\nabla_{\mathbf{x}}(\mathbf{b}^\top \mathbf{x}) &= \mathbf{b}, \\ \nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) &= (\mathbf{A} + \mathbf{A}^\top) \mathbf{x} \quad (= 2\mathbf{A} \mathbf{x} \text{ if } \mathbf{A} \text{ is symmetric}), \\ \nabla_{\mathbf{x}} \|\mathbf{A} \mathbf{x} - \mathbf{b}\|^2 &= 2\mathbf{A}^\top (\mathbf{A} \mathbf{x} - \mathbf{b}), \quad \nabla_{\mathbf{x}}^2 \|\mathbf{A} \mathbf{x} - \mathbf{b}\|^2 = 2\mathbf{A}^\top \mathbf{A}, \\ \nabla_{\mathbf{A}} \text{Tr}(\mathbf{A} \mathbf{B}) &= \mathbf{B}^\top, \quad \nabla_{\mathbf{A}} \log \det \mathbf{A} = \mathbf{A}^{-\top}.\end{aligned}$$

All four non-trivial ones are checked numerically against finite differences in `code/ml/c02_maths.py`.

Remark 2.2.1 — Trap. Writing $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{A} \mathbf{x}$ for a non-symmetric $\mathbf{A}$ is a common slip; the correct answer is $(\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$. When deriving by hand, check dimensions at every step: a gradient with respect to a d -vector is a d -vector, so terms like $\mathbf{x}^\top \mathbf{A}$ must be transposed.

§2.3 Probability

Prototypical example for this section: A test for a disease with 1% prevalence is 95% sensitive and 90% specific; a patient tests positive.

§2.3.i Bayes’ rule

$$\mathbb{P}(A | B) = \frac{\mathbb{P}(B | A)\mathbb{P}(A)}{\mathbb{P}(B)}, \quad \mathbb{P}(B) = \sum_k \mathbb{P}(B | A_k)\mathbb{P}(A_k).$$

In ML language: posterior $\propto$ likelihood $\times$ prior.

Example 2.3.1 (Base rates)

Prevalence 0.01, sensitivity $\mathbb{P}(+ | D) = 0.95$, specificity $\mathbb{P}(- | \neg D) = 0.90$. Then $\mathbb{P}(+) = 0.95 \cdot 0.01 + 0.10 \cdot 0.99 = 0.1085$ and

$$\mathbb{P}(D | +) = \frac{0.0095}{0.1085} = 0.0876.$$

Under 9%: most positives are false positives because healthy people vastly outnumber sick ones. This is the precision problem of every rare-event classifier

(Chapter 13).

§2.3.ii Expectation, variance, covariance

Expectation is linear always: $\mathbb{E}[aX + bY] = a\mathbb{E}X + b\mathbb{E}Y$, independent or not. $\text{Var } X = \mathbb{E}[X^2] - (\mathbb{E}X)^2$, $\text{Var}(aX + b) = a^2 \text{Var } X$, and

$$\text{Var}(X + Y) = \text{Var } X + \text{Var } Y + 2 \text{Cov}(X, Y), \quad \text{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}X \mathbb{E}Y.$$

For a vector, the **covariance matrix** $\Sigma = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top]$ is PSD and $\text{Var}(\mathbf{w}^\top \mathbf{x}) = \mathbf{w}^\top \Sigma \mathbf{w}$. A fair die has mean 3.5 and variance $35/12 = 2.9167$.

§2.3.iii Correlation and r^2

Definition 2.3.2. The (Pearson) **correlation** is $r = \text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1]$.

Covariance has units (rupees $\times$ kilograms) and changes when you rescale; correlation is unit-free and invariant to positive affine maps of either variable. It measures *linear* association only.

Example 2.3.3 (Covariance and correlation of five points)

$x = (2, 4, 6, 8, 10)$, $y = (3, 7, 5, 11, 14)$. Sample covariance (divide by $n - 1 = 4$) is 13.00; $s_x = 3.1623$, $s_y = 4.4721$, so $r = 13 / (3.1623 \cdot 4.4721) = 0.9192$ and $r^2 = 0.8450$. Rescaling x by 100 makes the covariance 1300 but leaves r at 0.9192.

Theorem 2.3.4 (r^2 is the R^2 of simple regression)

For least-squares regression of y on a single x with an intercept, the coefficient of determination equals the squared correlation: $R^2 = r^2$. (Proof in Chapter 3.)

So the OA question “the correlation between x and y is -0.8 ; what fraction of the variance of y is explained by a linear fit on x ?” has answer $(-0.8)^2 = 0.64$, i.e. 64%. The sign of r disappears.

Remark 2.3.5 — Trap. Three classics. (i) $r = 0$ does not mean independent: for x uniform on $[-1, 1]$ and $y = x^2$, y is a deterministic function of x yet $r = 0.0000$. (ii) $r^2 = 0.64$ does not mean “64% of points are predicted correctly”; it is a fraction of *variance*. (iii) $R^2 = r^2$ holds for *simple* linear regression with intercept; with several features R^2 is the squared correlation between y and $\hat{y}$, not with any one feature.

§2.3.iv Law of large numbers and the central limit theorem

Theorem 2.3.6 (LLN and CLT)

If $X_1, X_2, \dots$ are i.i.d. with mean μ and variance $\sigma^2 < \infty$, then the sample mean $\bar{X}_n \rightarrow \mu$ (LLN), and $\sqrt{n}(\bar{X}_n - \mu)/\sigma \rightarrow \mathcal{N}(0, 1)$ in distribution (CLT). In particular $\text{sd}(\bar{X}_n) = \sigma/\sqrt{n}$, the **standard error**.

The LLN is why empirical risk approximates risk; the CLT is why confidence intervals and z -tests (Chapter 14) work even for non-normal data. Figure 2.1 shows both for an exponential distribution, which is heavily skewed (skewness 1.98); means of 30 draws already have skewness 0.36.

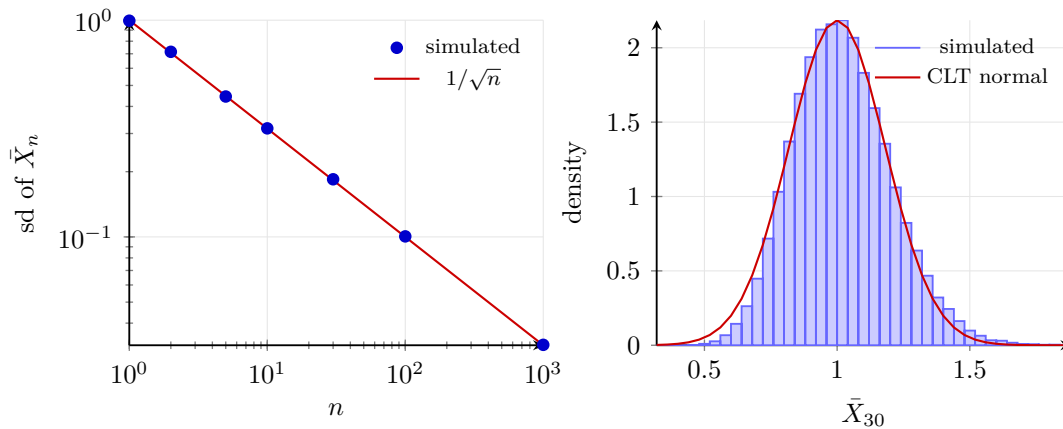


Figure 2.1: Means of Exponential(1) draws: the spread shrinks like $1/\sqrt{n}$ (left) and the shape becomes normal (right, $n = 30$, 50 000 repetitions).

§2.4 Information theory in five lines

Prototypical example for this section: Cross-entropy between the true label distribution and a classifier's predicted probabilities.

For distributions p, q over the same outcomes:

$$\text{entropy } H(p) = -\sum_i p_i \log p_i,$$

$$\text{cross-entropy } H(p, q) = -\sum_i p_i \log q_i = H(p) + \text{KL}(p \parallel q),$$

$$\text{KL divergence } \text{KL}(p \parallel q) = \sum_i p_i \log \frac{p_i}{q_i} \geq 0.$$

With $p = (0.5, 0.25, 0.25)$, $q = (0.4, 0.4, 0.2)$ in bits: $H(p) = 1.5000$, $H(p, q) = 1.5719$, $\text{KL}(p \parallel q) = 0.0719$ but $\text{KL}(q \parallel p) = 0.0781$ — KL is not symmetric and not a metric. Because $H(p)$ does not depend on the model q , minimising cross-entropy *is* minimising KL to the data distribution. Entropy reappears as a tree impurity (Chapter 9).

Remark 2.4.1 (Numerical stability) — $\log \sum_i e^{z_i}$ overflows for $z = (1000, 1001, 1002)$: numpy returns *inf*. Subtract the max first: $\log \sum e^{z_i} = m + \log \sum e^{z_i - m} = 1002.4076$. Every softmax and log-loss implementation does this.

§2.5 Maximum likelihood and MAP

Prototypical example for this section: A coin lands heads 7 times in 10; estimate p with and without a prior.

Definition 2.5.1. Given i.i.d. data and a model $p(\mathbf{x} \mid \boldsymbol{\theta})$, the **likelihood** is $L(\boldsymbol{\theta}) = \prod_i p(\mathbf{x}_i \mid \boldsymbol{\theta})$ and the **maximum likelihood estimate** (MLE) is $\arg \max_{\boldsymbol{\theta}} \sum_i \log p(\mathbf{x}_i \mid \boldsymbol{\theta})$. With a prior $p(\boldsymbol{\theta})$ the **maximum a posteriori** (MAP) estimate is $\arg \max_{\boldsymbol{\theta}} [\log p(\boldsymbol{\theta}) + \sum_i \log p(\mathbf{x}_i \mid \boldsymbol{\theta})]$.

We take logs because products of many small numbers underflow, sums are easier to differentiate, and log is monotone so the maximiser is unchanged.

Example 2.5.2 (Bernoulli MLE and MAP)

$\ell(p) = 7 \log p + 3 \log(1 - p)$; $\ell'(p) = 7/p - 3/(1 - p) = 0$ gives $\hat{p}_{\text{MLE}} = 7/10 = 0.70$. With a Beta(3, 3) prior (two pseudo-heads and two pseudo-tails) the posterior is Beta(10, 6), whose mode is $\hat{p}_{\text{MAP}} = (7 + 2)/(10 + 4) = 0.6429$ and whose mean is $10/16 = 0.6250$. The prior pulls the estimate towards $1/2$; with more data the pull fades. Laplace smoothing in naive Bayes (Chapter 6) is exactly this.

§2.5.i Three losses that are really likelihoods

1. **Least squares = Gaussian MLE.** If $y_i = \mathbf{w}^\top \mathbf{x}_i + \varepsilon_i$ with $\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$, then $-\log L(\mathbf{w}) = \frac{1}{2\sigma^2} \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \frac{n}{2} \log(2\pi\sigma^2)$, so maximising likelihood minimises the sum of squared errors.
2. **Ridge = MAP with a Gaussian prior.** Add $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I})$: $-\log p(\mathbf{w} | \mathcal{D}) = \frac{1}{2\sigma^2} \sum (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \frac{1}{2\tau^2} \|\mathbf{w}\|^2 + c$, i.e. ridge with $\lambda = \sigma^2/\tau^2$. A Laplace prior gives the lasso. A tight prior (small τ) means large λ .
3. **Cross-entropy = categorical MLE.** If the model outputs probabilities $\hat{p}_{ik}$ and y_i is one-hot, $-\log L = -\sum_i \sum_k y_{ik} \log \hat{p}_{ik}$, the log-loss of logistic and softmax regression (Chapter 5).

Remark 2.5.3 — Trap. The MLE of a Gaussian variance divides by n , not $n - 1$, and is biased. For the sample (4, 7, 5, 9, 5): mean 6.0, MLE variance 3.20, unbiased variance 4.00. `numpy.var` defaults to `ddof=0` (the MLE); `pandas` defaults to `ddof=1`. MLEs are typically consistent and asymptotically efficient, not necessarily unbiased.

§2.6 Convexity

Prototypical example for this section: Why $|x|$ is convex, $\sin x$ is not, and the 0-1 loss is not.

Definition 2.6.1. A set C is **convex** if the segment between any two of its points stays in C . A function f on a convex set is **convex** if for all $\mathbf{x}, \mathbf{y}$ and $t \in [0, 1]$

$$f(t\mathbf{x} + (1 - t)\mathbf{y}) \leq tf(\mathbf{x}) + (1 - t)f(\mathbf{y}) :$$

the chord lies on or above the graph. **Strictly convex:** strict inequality for $\mathbf{x} \neq \mathbf{y}$, $t \in (0, 1)$.

Theorem 2.6.2 (Tests)

For differentiable f : convex $\iff f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x})$ (every tangent is a global under-estimator). For twice-differentiable f : convex $\iff \nabla^2 f \succeq 0$ everywhere; $\nabla^2 f \succ 0$ everywhere implies strictly convex.

Theorem 2.6.3 (Why we care)

For a convex function every local minimum is a global minimum, and the set of minimisers is convex. A strictly convex function has at most one minimiser.

§2.6.i Building convex functions

Convexity is preserved by: non-negative weighted sums; composition with an affine map ($f(\mathbf{Ax} + \mathbf{b})$); and the *pointwise maximum* (or supremum) of convex functions. It is **not** preserved by the pointwise minimum, by products, or by differences.

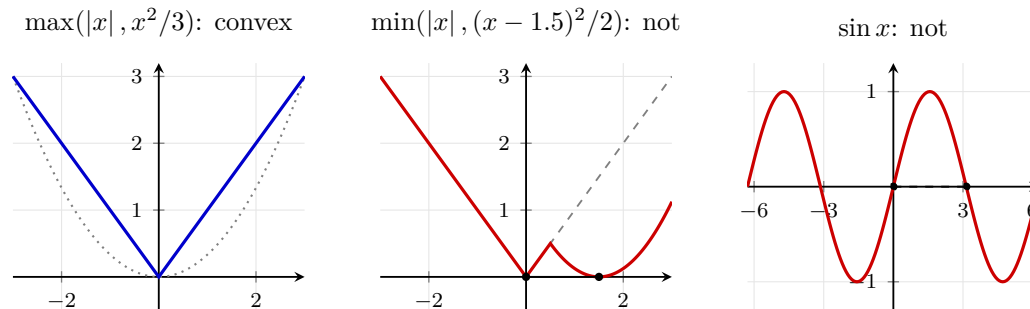


Figure 2.2: The maximum of convex functions is convex; the minimum generally is not (two local minima, at 0 and 1.5). For $\sin x$ the chord from 0 to π lies *below* the graph.

Example 2.6.4 (OA: which are convex?)

Each of these has appeared as a “which statements are true” option.

- $|x|$: convex (a max of x and $-x$, both linear), but not differentiable at 0 and not strictly convex.
- $\sin x$ on $\mathbb{R}$: not convex. Chord test on $[0, \pi]$: the midpoint of the chord is 0, but $\sin(\pi/2) = 1 > 0$. (It is convex on $[\pi, 2\pi]$ where $\sin'' = -\sin \geq 0$, which is why a question may restrict the domain.)
- $e^x, x^2, -\log x, x \log x, \|\mathbf{x}\|, \max_i x_i, \log \sum e^{x_i}$: convex. $\log x, \sqrt{x}$: concave.
- $\max(f, g)$ for convex f, g : convex. $\min(f, g)$: not in general (Figure 2.2).
- Jensen’s inequality $f(\mathbb{E}X) \leq \mathbb{E}f(X)$ for convex f : with X uniform on $\{1, 2, 6\}$ and $f(x) = x^2$, $(\mathbb{E}X)^2 = 9 \leq \mathbb{E}X^2 = 13.67$; this is just $\text{Var } X \geq 0$.

§2.6.ii Why the 0-1 loss is not convex, and what we do about it

Write a binary classifier as $\text{sign}(f(\mathbf{x}))$ with labels $y \in \{-1, +1\}$ and **margin** $m = yf(\mathbf{x})$. The 0-1 loss is $\mathbb{1}[m \leq 0]$, a step. Chord test: at $m = -0.5$ the loss is 1, at $m = 0.5$ it is 0, the chord’s midpoint is 0.5, but the loss at $m = 0$ is 1. It is also flat almost everywhere, so its gradient is zero almost everywhere and gradient methods get no signal; minimising it exactly over linear classifiers is NP-hard in general.

So we minimise a convex **surrogate loss** that upper-bounds the step: the **hinge** $\max(0, 1-m)$ (SVM), the **logistic** loss $\log_2(1+e^{-m})$ (logistic regression), the **exponential**

e^{-m} (AdaBoost), or the squared loss $(1 - m)^2$ (which also penalises being “too right”, a reason it is a poor classification loss).

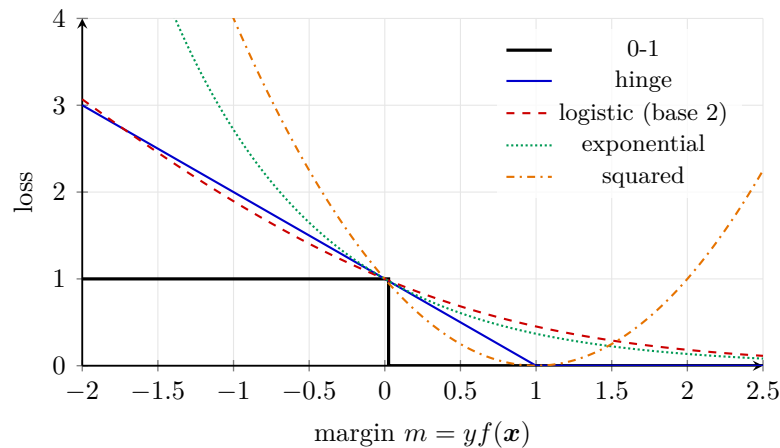


Figure 2.3: The 0-1 loss and its convex surrogates as functions of the margin. All surrogates upper-bound the step; only the squared loss rises again for confident correct predictions.

Remark 2.6.5 — Trap. “Convex loss $\Rightarrow$ convex training objective.” Only if the model is linear in its parameters. Log-loss of a neural network is non-convex in the weights even though log-loss is convex in the logits: composition of a convex function with a non-affine map breaks convexity.

Max of convex is convex, min is not; the 0-1 loss is a non-convex step with zero gradient, so we optimise convex surrogates that bound it from above.

§2.7 Lagrange multipliers and KKT

Prototypical example for this section: Maximise xy subject to $x + y = 10$; then minimise $(x - 2)^2$ subject to $x \leq 1$.

§2.7.i Equality constraints

To optimise $f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$, form the **Lagrangian** $\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$ and set $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{0}$, $g = 0$. Geometrically, at the optimum ∇f is parallel to ∇g : you cannot improve f while staying on the constraint surface.

Example 2.7.1

Maximise xy with $x + y = 10$: $\nabla(xy) = (y, x) = \lambda(1, 1)$ forces $x = y = 5$ (checked with SciPy’s SLSQP). A more ML-flavoured one: the maximum-entropy distribution on $\{1, 2, 3\}$ with mean 2.5 has the exponential-family form $p_i \propto e^{tx_i}$; solving for t gives $p = (0.116, 0.268, 0.616)$, matching a numerical constrained optimiser. The same calculation with an unconstrained mean gives the uniform distribution, and with a variance constraint on $\mathbb{R}$ gives the Gaussian.

§2.7.ii Inequality constraints: KKT

For $\min f(\mathbf{x})$ s.t. $g_j(\mathbf{x}) \leq 0$, $h_k(\mathbf{x}) = 0$, with Lagrangian $f + \sum_j \mu_j g_j + \sum_k \lambda_k h_k$, the **Karush–Kuhn–Tucker** (KKT) conditions are

1. stationarity: $\nabla f + \sum \mu_j \nabla g_j + \sum \lambda_k \nabla h_k = \mathbf{0}$;
2. primal feasibility: $g_j \leq 0$, $h_k = 0$;
3. dual feasibility: $\mu_j \geq 0$;
4. **complementary slackness**: $\mu_j g_j(\mathbf{x}) = 0$ for each j .

For convex problems satisfying a mild regularity condition (Slater: a strictly feasible point exists), KKT is necessary and sufficient.

Example 2.7.2 (An active constraint)

$\min(x-2)^2$ s.t. $x \leq 1$. Unconstrained the answer would be 2, which is infeasible, so the constraint is active: $x^* = 1$. Stationarity $2(x-2) + \mu = 0$ gives $\mu = 2 > 0$. Complementary slackness reads: either the constraint is tight ($x = 1$) or its multiplier is zero. In SVMs (Chapter 8) this is exactly why only the *support vectors* have non-zero α_i .

Multipliers price constraints; complementary slackness says a constraint either binds or costs nothing.

§2.8 A few problems to think about

Problem 2A (r^2 from r). The correlation between hours studied and marks is -0.3 in one batch and 0.6 in another. In each, what fraction of the variance of marks is explained by a straight-line fit on hours? Which batch has the “stronger” linear relationship?

hint · solution

Problem 2B (Which are convex?). Decide convexity on the stated domain: (a) $f(x) = \max(x^2, 2x+1)$; (b) $f(x) = \min(x^2, 1)$; (c) $f(\mathbf{w}) = \sum_i \log(1 + e^{-y_i \mathbf{w}^\top \mathbf{x}_i}) + \lambda \|\mathbf{w}\|_1$; (d) $f(x) = x^3$ on $\mathbb{R}$; (e) $f(x) = x^3$ on $[0, \infty)$; (f) $f(x, y) = xy$.

hint · solution

Problem 2C (Gradient of a quadratic form). Let $f(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{A} \mathbf{w} - \mathbf{b}^\top \mathbf{w} + c$ with $\mathbf{A}$ symmetric PD. Find the minimiser and the minimum value. What goes wrong if $\mathbf{A}$ has a zero eigenvalue?

hint · solution

Problem 2D (Bayes with two tests). In the disease example (prevalence 1%, sensitivity 95%, specificity 90%) the patient takes a second, conditionally independent test with the same characteristics and is positive again. What is $\mathbb{P}(D \mid ++)$?

hint · solution

Problem 2E (MAP shrinkage). A click-through rate is estimated from 2 clicks in 3 impressions. Compare the MLE with the MAP under a Beta(2, 20) prior. Which would you show on a dashboard ranking ads?

hint · solution

Problem 2F (KKT in one dimension). Solve $\min_x (x - a)^2$ subject to $0 \leq x \leq 1$ for any real a using KKT, and identify which multiplier is positive in each case.

[hint](#) · [solution](#)



II

Supervised Learning

Part II: Contents

3	Linear Regression	33
3.1	Least squares and the normal equations	33
3.2	The projection and MLE views	34
3.3	Five points by hand	35
3.4	Gradient descent, traced	36
3.5	R^2 and adjusted R^2	37
3.6	Assumptions, diagnostics, multicollinearity	38
3.7	Polynomial features and robust losses	39
3.8	A few problems to think about	40
4	Bias, Variance and Regularisation	41
4.1	The decomposition	41
4.2	Ridge	41
4.3	Lasso and elastic net	41
4.4	The augmented error	41
4.5	Early stopping and double descent	41
5	Logistic Regression and GLMs	43
5.1	From odds to the sigmoid	43
5.2	Log-loss from MLE	43
5.3	Newton and IRLS	43
5.4	Softmax regression and multiclass	43
5.5	Imbalance and thresholds	43
5.6	The perceptron and XOR	43
6	Generative Classifiers	45
6.1	Naive Bayes	45
6.2	Laplace smoothing	45
6.3	LDA and QDA	45
7	k-Nearest Neighbours and Distances	47
7.1	The algorithm	47
7.2	Distances	47
7.3	Choosing k	47
7.4	The curse of dimensionality	47
7.5	KD-trees and approximate search	47
8	Support Vector Machines	49
8.1	Margins	49
8.2	Hard and soft margin	49
8.3	Duality	49
8.4	Kernels	49
8.5	SVR and SMO	49
8.6	Convex surrogate losses	49
9	Decision Trees	51
9.1	Impurity	51
9.2	A split by hand	51
9.3	Growing a tree	51
9.4	Regression trees	51
9.5	Pruning	51
9.6	What trees are good for	51
10	Ensembles	53
10.1	The bootstrap and 63.2 percent	53

10.2	Bagging and random forests	53
10.3	Feature importance and its traps	53
10.4	AdaBoost	53
10.5	Gradient boosting	53
10.6	XGBoost	53
10.7	LightGBM and CatBoost	53
10.8	Stacking and the bias-variance story	53

3 Linear Regression

Linear regression is the one model every interviewer assumes you can derive in three ways (calculus, geometry, probability), fit by hand in an OA, and criticise in a design round. It is also the template for everything else: a hypothesis class, a loss, a closed form or a gradient, and a list of assumptions whose violation you should be able to diagnose.

§3.1 Least squares and the normal equations

Prototypical example for this section: Predict flat price from area: $\hat{y} = w_0 + w_1 \cdot \text{area}$.

§3.1.i The model

Stack n examples as rows of the **design matrix** $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$, whose first column is all ones (the intercept), and the targets as $\mathbf{y} \in \mathbb{R}^n$. The model is $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ and **ordinary least squares** (OLS) minimises the **residual sum of squares**

$$\text{RSS}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2.$$

“Linear” means linear *in the parameters* $\mathbf{w}$: $y = w_0 + w_1x + w_2x^2 + w_3 \log x$ is still linear regression.

§3.1.ii The closed form

Using $\nabla_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ from **Chapter 2** and setting it to zero gives the **normal equations**

$$\mathbf{X}^\top \mathbf{X} \hat{\mathbf{w}} = \mathbf{X}^\top \mathbf{y} \quad \implies \quad \hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

whenever $\mathbf{X}^\top \mathbf{X}$ is invertible (i.e. $\mathbf{X}$ has full column rank). The Hessian $2\mathbf{X}^\top \mathbf{X}$ is PSD, so RSS is convex and any stationary point is a global minimum; if $\mathbf{X}$ has full column rank the Hessian is PD and the minimiser is unique.

Remark 3.1.1 (How it is actually computed) — Nobody inverts $\mathbf{X}^\top \mathbf{X}$. Forming it squares the condition number: for an intercept column plus a feature ranging over $[1000, 1001]$, $\kappa(\mathbf{X}) = 3.4 \times 10^6$ but $\kappa(\mathbf{X}^\top \mathbf{X}) = 1.2 \times 10^{13}$, so you lose about twice as many digits. Libraries use a QR decomposition ($\mathbf{X} = \mathbf{Q}\mathbf{R}$, then solve $\mathbf{R}\mathbf{w} = \mathbf{Q}^\top \mathbf{y}$) or the SVD (`numpy.linalg.lstsq`, `scikit-learn`). On a 200-row, 3-feature example the normal equations, QR, `lstsq` and `LinearRegression` all give $\hat{\mathbf{w}} = (2.982, 1.955, -1.048, 0.527)$ to machine precision (`code/ml/c03_linreg.py`). Cost: $O(nd^2 + d^3)$; for very large d or streaming data, use gradient descent instead.

Remark 3.1.2 — **Trap.** “The normal equations fail when features are correlated.” They fail when features are *perfectly* collinear (or $n < d + 1$), because $\mathbf{X}^\top \mathbf{X}$ is singular. Strong but imperfect correlation still gives a unique solution, only an unstable one (§3.6). Centring and scaling features do not change the fitted values

of OLS, only the coefficients' scale and the numerical conditioning.

§3.2 The projection and MLE views

Prototypical example for this section: $\hat{\mathbf{y}}$ is the shadow of $\mathbf{y}$ on the plane spanned by the feature columns.

§3.2.i Geometry

$\mathbf{X}\mathbf{w}$ ranges over the column space $\mathcal{C}(\mathbf{X})$, a $(d + 1)$ -dimensional subspace of $\mathbb{R}^n$. The closest point to $\mathbf{y}$ in it is the orthogonal projection $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ with $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ (the **hat matrix**: it puts the hat on $\mathbf{y}$). The normal equations say $\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$: the residual vector is orthogonal to every column of $\mathbf{X}$. Two corollaries an interviewer can ask:

- With an intercept, residuals sum to zero (orthogonal to the ones column) and the fitted line passes through $(\bar{x}, \bar{y})$.
- Residuals are uncorrelated with every feature and with $\hat{\mathbf{y}}$ in the sample. A residual plot that shows a trend against $\hat{\mathbf{y}}$ therefore signals *non-linearity*, not a coding bug.

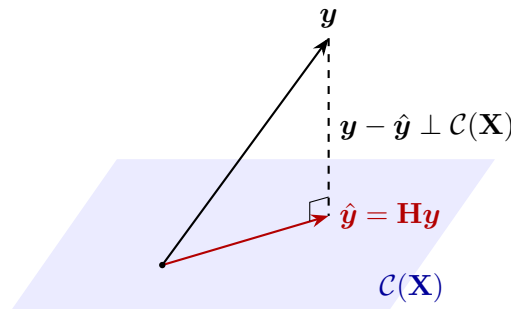


Figure 3.1: Least squares projects $\mathbf{y}$ onto the column space of $\mathbf{X}$; the residual is perpendicular to it.

§3.2.ii Probability

Assume $y_i = \mathbf{w}^\top \mathbf{x}_i + \varepsilon_i$ with $\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$. The log-likelihood is $-\frac{1}{2\sigma^2} \text{RSS}(\mathbf{w}) - \frac{n}{2} \log(2\pi\sigma^2)$, so the MLE of $\mathbf{w}$ is OLS (as derived in **Chapter 2**), and the MLE of σ^2 is RSS/n (biased; the unbiased estimate divides by $n - d - 1$). Squared loss is therefore the natural loss when noise is Gaussian; with Laplace noise the MLE is least *absolute* deviations, which estimates a conditional median.

Theorem 3.2.1 (Gauss–Markov)

If $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$ and $\text{Cov}(\boldsymbol{\varepsilon}) = \sigma^2 \mathbf{I}$ (zero mean, equal variance, uncorrelated — normality *not* required), then OLS is the **BLUE**: the best (minimum-variance) linear unbiased estimator of $\mathbf{w}$. Its covariance is $\sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$.

“Best among *linear unbiased*” is the catch: ridge is biased and can have lower mean squared error (Chapter 4).

OLS = calculus (normal equations) = geometry (projection) = statistics (Gaussian MLE).

§3.3 Five points by hand

Prototypical example for this section: The OA question: “fit $y = b_0 + b_1x$ to these five points”.

For one feature the normal equations reduce to

$$\hat{b}_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} = r \frac{s_y}{s_x}, \quad \hat{b}_0 = \bar{y} - \hat{b}_1 \bar{x}.$$

The middle form is what you use with raw sums when the means are messy; the centred form is faster when $\bar{x}$ is an integer.

Example 3.3.1 (The OA fit)

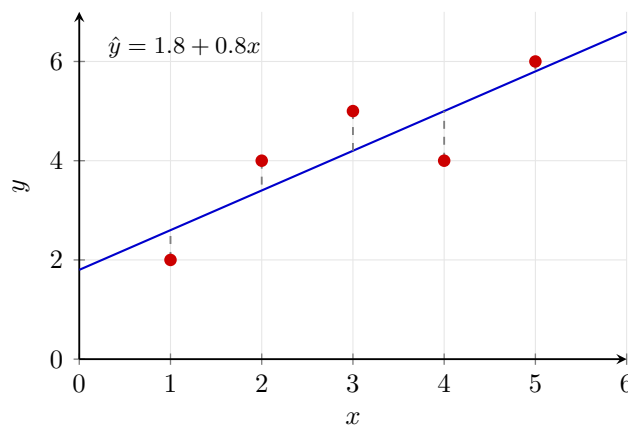
Data: $x = (1, 2, 3, 4, 5)$, $y = (2, 4, 5, 4, 6)$. Raw sums: $\sum x = 15$, $\sum y = 21$, $\sum xy = 71$, $\sum x^2 = 55$.

$$\hat{b}_1 = \frac{5 \cdot 71 - 15 \cdot 21}{5 \cdot 55 - 15^2} = \frac{40}{50} = 0.80, \quad \hat{b}_0 = 4.2 - 0.8 \cdot 3 = 1.80.$$

Centred check: $\sum(x - \bar{x})(y - \bar{y}) = 8$ and $\sum(x - \bar{x})^2 = 10$, ratio 0.8. Fitted values and residuals:

x	1	2	3	4	5
y	2	4	5	4	6
$\hat{y}$	2.6	3.4	4.2	5.0	5.8
$y - \hat{y}$	-0.6	+0.6	+0.8	-1.0	+0.2

The residuals sum to 0.000 and $\sum x_i(y_i - \hat{y}_i) = 0.000$, as the normal equations demand. RSS = 2.40, TSS = $\sum(y - \bar{y})^2 = 8.8$, so $R^2 = 1 - 2.4/8.8 = 0.7273$, which equals r^2 with $r = 0.8528$. Prediction at $x = 6$: 6.60. (Checked against `LinearRegression`, `numpy.polyfit` and `scipy.stats.linregress`.)



Example 3.3.2 (Is the slope significant?)

Residual variance $s^2 = \text{RSS}/(n-2) = 0.8$; standard error $\text{SE}(\hat{b}_1) = \sqrt{s^2/\sum(x-\bar{x})^2} = 0.2828$; $t = 0.8/0.2828 = 2.828$ on 3 degrees of freedom, two-sided $p = 0.0663$. With five points, an R^2 of 0.73 is *not* significant at the 5% level. Small samples make large R^2 cheap.

Remark 3.3.3 — Trap. Regressing x on y does *not* give slope $1/\hat{b}_1$. It gives $r s_x/s_y$, and the product of the two slopes is $r^2 \leq 1$. Both lines pass through $(\bar{x}, \bar{y})$ and coincide only if $|r| = 1$. Also: the slope has units of y per unit of x , so “a bigger coefficient means a more important feature” is meaningless unless features are on a common scale.

§3.4 Gradient descent, traced

Prototypical example for this section: Three gradient steps on the five points, by hand.

For MSE $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$ the gradient is $\frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$ and the update is $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$. In coordinates, with $e_i = \hat{y}_i - y_i$:

$$w_0 \leftarrow w_0 - \eta \cdot \frac{2}{n} \sum e_i, \quad w_1 \leftarrow w_1 - \eta \cdot \frac{2}{n} \sum e_i x_i.$$

Example 3.4.1 (The first iterations, $\eta = 0.05$, from $\mathbf{w} = \mathbf{0}$)

t	w_0	w_1	MSE	$\partial J/\partial w_0$	$\partial J/\partial w_1$
0	0.0000	0.0000	19.4000	-8.4000	-28.4000
1	0.4200	1.4200	1.4792	0.9600	5.3600
2	0.3720	1.1520	0.8662	-0.7440	-0.8240
3	0.4092	1.1932	0.8338	-0.4224	0.3056

Step 0: all predictions are 0, errors are $-y$, so $\partial J/\partial w_0 = -2 \cdot 21/5 = -8.4$ and $\partial J/\partial w_1 = -2 \cdot 71/5 = -28.4$; the step moves to $(0.42, 1.42)$. After 500 steps $\mathbf{w} = (1.7997, 0.8001)$, the closed form to three decimals, and the MSE floor is $\text{RSS}/n = 0.4800$.

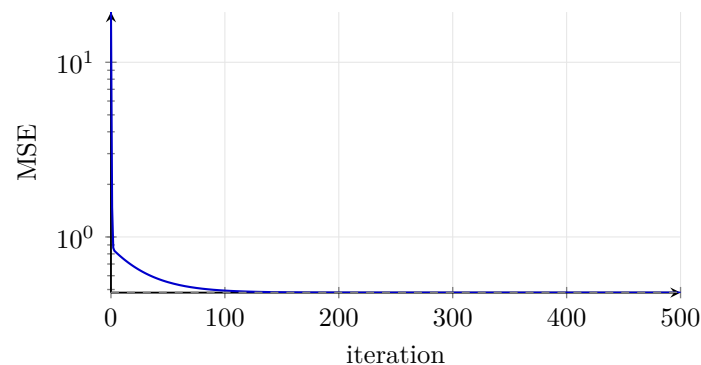


Figure 3.2: GD on the five points: a fast drop along the steep direction, then slow progress along the shallow one (the intercept), typical of an ill-conditioned quadratic.

On a quadratic, GD converges iff $\eta < 2/\lambda_{\max}$ of the Hessian $\frac{2}{n} \mathbf{X}^\top \mathbf{X}$; here that bound

is 0.0845, so $\eta = 0.05$ is safe and $\eta = 0.1$ would diverge (Chapter 16). Unscaled features make $\lambda_{\max}$ huge, forcing a tiny η and slow convergence along the other directions: the practical reason to standardise before gradient descent, even though OLS's solution does not care.

§3.5 R^2 and adjusted R^2

Prototypical example for this section: Adding twenty columns of random noise raises R^2 from 0.64 to 0.83.

Definition 3.5.1. $R^2 = 1 - \text{RSS} / \text{TSS}$ with $\text{TSS} = \sum (y_i - \bar{y})^2$: the fraction of variance of y explained by the model, relative to predicting $\bar{y}$. With p features (excluding the intercept),

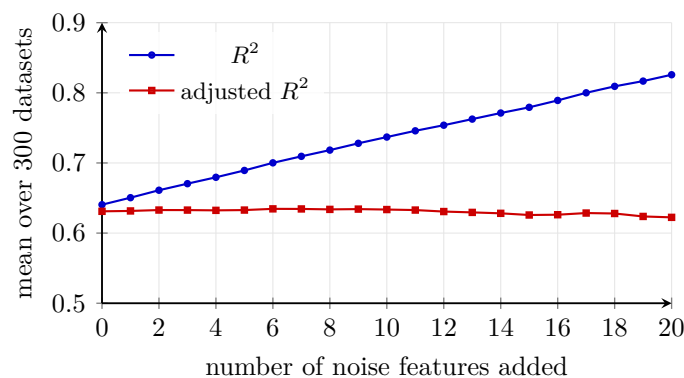
$$R^2_{\text{adj}} = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}.$$

Facts worth stating crisply:

- On the training data with an intercept, $0 \leq R^2 \leq 1$, and R^2 never decreases when you add a feature (the old fit is still available).
- On *test* data R^2 can be negative: the model is worse than predicting the training mean. `sklearn's r2_score` returns such values.
- Adjusted R^2 rises when a new feature's $|t| > 1$ and falls otherwise; it penalises features but is not a substitute for validation.

Example 3.5.2 (Junk features)

$n = 40$, one real feature, then $k = 0, \dots, 20$ added columns of pure noise, averaged over 300 datasets. R^2 climbs from 0.641 to 0.826; adjusted R^2 stays flat ($0.631 \rightarrow 0.622$). OA version: $R^2 = 0.80$, $n = 50$, $p = 4$ gives $R^2_{\text{adj}} = 1 - 0.2 \cdot 49/45 = 0.7822$.



Remark 3.5.3 — Trap. “High R^2 means the model is correct” and “low R^2 means it is useless” are both wrong. A curved relationship fitted by a line can have $R^2 = 0.9$ and systematic residuals; a noisy but real effect (returns on a stock) can have $R^2 = 0.02$ and be valuable. R^2 also depends on the spread of x in the sample: restrict the range and R^2 drops with the same underlying slope.

§3.6 Assumptions, diagnostics, multicollinearity

Prototypical example for this section: Two nearly identical features — area in square feet and in square metres rounded differently — get coefficients +300 and −290.

§3.6.i The assumptions, and what breaks when each fails

Assumption	Diagnostic	If violated
Linearity of $\mathbb{E}[y \mid \mathbf{x}]$	residuals vs fitted show a curve	biased predictions; add features/transforms or a non-linear model
Independent errors	residuals vs time/order, Durbin–Watson	standard errors too small; use time-series models or clustered errors
Homoscedasticity (constant variance)	funnel-shaped residual plot	OLS still unbiased but SEs wrong; log-transform y , weighted LS, robust SEs
Normal errors	Q–Q plot of residuals	only matters for small-sample tests and intervals
No perfect collinearity	rank of $\mathbf{X}$, VIF	no unique solution; drop/combine features, ridge
No influential outliers	leverage h_{ii} , Cook’s distance	a single point drags the line; robust losses

Remark 3.6.1 — Trap. “Linear regression requires x and y to be normally distributed.” No assumption is made about the distribution of x at all, and the normality assumption (needed only for exact t/F inference) is on the *errors*, not on y .

§3.6.ii Multicollinearity and VIF

When features are highly correlated, $\mathbf{X}^\top \mathbf{X}$ is nearly singular, $(\mathbf{X}^\top \mathbf{X})^{-1}$ has huge entries, and since $\text{Cov}(\hat{\mathbf{w}}) = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$ the individual coefficients have huge variance. Predictions inside the data range are fine; *interpretation* of individual coefficients is not.

Definition 3.6.2. The **variance inflation factor** of feature j is $\text{VIF}_j = 1/(1 - R_j^2)$, where R_j^2 is the R^2 of regressing feature j on all the other features. Equivalently, VIF_j is the j -th diagonal entry of the inverse of the feature correlation matrix. Rules of thumb: above 5 worrying, above 10 serious.

Example 3.6.3 (VIF and unstable coefficients)

Features a, b, c with $\text{Corr}(a, b) = 0.950$ and c independent. VIFs are (10.34, 10.32, 1.00) (computed both by auxiliary regressions and from the inverse correlation matrix). Fit $y = 1 + a + b + c + \varepsilon$ on 500 bootstrap resamples: the standard deviation of $\hat{w}_a$ is 0.196, about 3.7 times that of $\hat{w}_c$ (0.053), roughly $\sqrt{\text{VIF}}$ as theory predicts. But the *sum* $\hat{w}_a + \hat{w}_b$ has standard deviation only 0.062: the data pin down the combined effect, not how to split it.

Remark 3.6.4 — Trap. Multicollinearity does not bias OLS and does not hurt prediction on data like the training data. It inflates the variance of individual coefficients, flips signs, and makes “feature importance from coefficients” meaningless. Remedies: drop one of a correlated pair, combine them, use ridge (which splits weight between them), or PCA regression.

§3.6.iii Interpreting coefficients

$\hat{w}_j$ is the change in predicted y per unit change in x_j *holding the other features fixed*. With $\log y$ as the target, $100\hat{w}_j$ is approximately the percentage change in y per unit of x_j ; with both logged, $\hat{w}_j$ is an elasticity. A dummy variable’s coefficient is the shift relative to the omitted (reference) category; including all K dummies plus an intercept is the **dummy variable trap** (perfect collinearity).

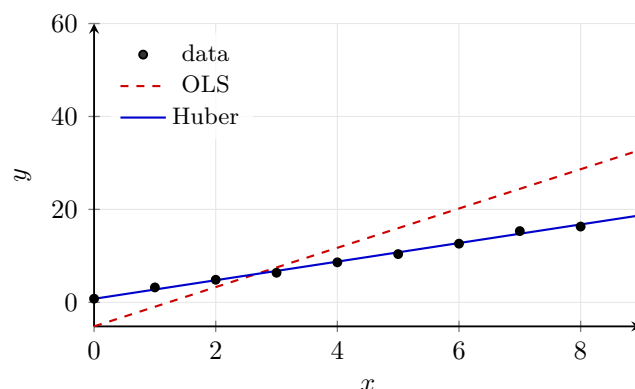
§3.7 Polynomial features and robust losses

Prototypical example for this section: Fit $y = w_0 + w_1x + w_2x^2$ with the same OLS machinery.

Replacing $\mathbf{x}$ by a feature map $\phi(\mathbf{x})$ (powers, interactions, splines, logs) keeps the problem linear in $\mathbf{w}$, so all of the above applies. The cost is dimension: all monomials of degree $\leq D$ in p variables number $\binom{p+D}{D}$. For $p = 10$: degree 2 gives 66 features and degree 3 gives 286; $p = 100$ at degree 2 already gives 5151 (counts checked against `PolynomialFeatures`). High degrees overfit and extrapolate wildly (Figure 1.1); scale x first, because x^9 for $x \sim 1000$ wrecks conditioning. Kernels (Chapter 8) are a way to use enormous feature maps without forming them.

Example 3.7.1 (One outlier)

Ten points on $y \approx 2x + 1$, with the last y replaced by 60. The OLS slope becomes 4.23; the Huber loss (quadratic for small residuals, linear for large ones) gives 2.00. Squared loss lets one point with a large residual dominate the fit.



Linear in the parameters, not in the inputs: feature maps buy flexibility, squared loss buys a closed form and sensitivity to outliers.

§3.8 A few problems to think about

Problem 3A (Fit a line). Fit $y = b_0 + b_1x$ to $(1, 1), (2, 3), (3, 2), (4, 5), (5, 4)$. Report b_0, b_1, R^2 , and the prediction at $x = 7$.

[hint · solution](#)

Problem 3B (Slope from summary statistics). For 100 students, the correlation between study hours and score is 0.6, the standard deviation of hours is 2 and of scores is 10. Mean hours 5, mean score 60. Give the regression line of score on hours and the predicted score for 8 hours. What is the slope of the regression of hours on score?

[hint · solution](#)

Problem 3C (Duplicated feature). You accidentally include the same feature twice, $x_2 = x_1$. What happens to (a) the normal equations, (b) `sklearn`'s `LinearRegression`, (c) the predictions, (d) ridge?

[hint · solution](#)

Problem 3D (Adjusted R^2). Model A has 3 features and $R^2 = 0.70$; model B has 10 features and $R^2 = 0.74$; $n = 60$. Which has higher adjusted R^2 ?

[hint · solution](#)

Problem 3E (Residual orthogonality). Prove that for OLS with an intercept, $\sum_i (y_i - \hat{y}_i) = 0$ and $\sum_i \hat{y}_i (y_i - \hat{y}_i) = 0$. Deduce $\text{TSS} = \text{RSS} + \text{ESS}$ where $\text{ESS} = \sum (\hat{y}_i - \bar{y})^2$.

[hint · solution](#)

Problem 3F (Learning-rate bound). For $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$, show that gradient descent converges for every starting point iff $0 < \eta < 2/\lambda_{\max}$ where $\lambda_{\max}$ is the largest eigenvalue of $\frac{2}{n} \mathbf{X}^\top \mathbf{X}$ (assume full column rank).

[hint · solution](#)

4 Bias, Variance and Regularisation

§4.1 The decomposition

[STUB: The decomposition]

§4.2 Ridge

[STUB: Ridge]

§4.3 Lasso and elastic net

[STUB: Lasso and elastic net]

§4.4 The augmented error

[STUB: The augmented error]

§4.5 Early stopping and double descent

[STUB: Early stopping and double descent]

5 Logistic Regression and GLMs

§5.1 From odds to the sigmoid

[STUB: From odds to the sigmoid]

§5.2 Log-loss from MLE

[STUB: Log-loss from MLE]

§5.3 Newton and IRLS

[STUB: Newton and IRLS]

§5.4 Softmax regression and multiclass

[STUB: Softmax regression and multiclass]

§5.5 Imbalance and thresholds

[STUB: Imbalance and thresholds]

§5.6 The perceptron and XOR

[STUB: The perceptron and XOR]

6 Generative Classifiers

§6.1 Naive Bayes

[STUB: Naive Bayes]

§6.2 Laplace smoothing

[STUB: Laplace smoothing]

§6.3 LDA and QDA

[STUB: LDA and QDA]

7 k-Nearest Neighbours and Distances

§7.1 The algorithm

[STUB: The algorithm]

§7.2 Distances

[STUB: Distances]

§7.3 Choosing k

[STUB: Choosing k]

§7.4 The curse of dimensionality

[STUB: The curse of dimensionality]

§7.5 KD-trees and approximate search

[STUB: KD-trees and approximate search]

8 Support Vector Machines

§8.1 Margins

[STUB: Margins]

§8.2 Hard and soft margin

[STUB: Hard and soft margin]

§8.3 Duality

[STUB: Duality]

§8.4 Kernels

[STUB: Kernels]

§8.5 SVR and SMO

[STUB: SVR and SMO]

§8.6 Convex surrogate losses

[STUB: Convex surrogate losses]

9 Decision Trees

§9.1 Impurity

[STUB: Impurity]

§9.2 A split by hand

[STUB: A split by hand]

§9.3 Growing a tree

[STUB: Growing a tree]

§9.4 Regression trees

[STUB: Regression trees]

§9.5 Pruning

[STUB: Pruning]

§9.6 What trees are good for

[STUB: What trees are good for]

10 Ensembles

§10.1 The bootstrap and 63.2 percent

[STUB: The bootstrap and 63.2 percent]

§10.2 Bagging and random forests

[STUB: Bagging and random forests]

§10.3 Feature importance and its traps

[STUB: Feature importance and its traps]

§10.4 AdaBoost

[STUB: AdaBoost]

§10.5 Gradient boosting

[STUB: Gradient boosting]

§10.6 XGBoost

[STUB: XGBoost]

§10.7 LightGBM and CatBoost

[STUB: LightGBM and CatBoost]

§10.8 Stacking and the bias-variance story

[STUB: Stacking and the bias-variance story]

III

Learning Without (Enough) Labels

Part III: Contents

11	Unsupervised Learning	57
11.1	k-means	57
11.2	Hierarchical clustering	57
11.3	DBSCAN	57
11.4	Gaussian mixtures and EM	57
11.5	PCA	57
11.6	ICA, t-SNE and UMAP	57
11.7	Anomaly detection	57
11.8	Association rules	57
12	Semi- and Self-Supervised Learning	59
12.1	Assumptions	59
12.2	Self-training	59
12.3	Co-training and label propagation	59
12.4	Consistency regularisation	59
12.5	When unlabelled data hurts	59
12.6	Contrastive learning in one page	59

11 Unsupervised Learning

§11.1 k-means

[STUB: k-means]

§11.2 Hierarchical clustering

[STUB: Hierarchical clustering]

§11.3 DBSCAN

[STUB: DBSCAN]

§11.4 Gaussian mixtures and EM

[STUB: Gaussian mixtures and EM]

§11.5 PCA

[STUB: PCA]

§11.6 ICA, t-SNE and UMAP

[STUB: ICA, t-SNE and UMAP]

§11.7 Anomaly detection

[STUB: Anomaly detection]

§11.8 Association rules

[STUB: Association rules]

12 Semi- and Self-Supervised Learning

§12.1 Assumptions

[STUB: Assumptions]

§12.2 Self-training

[STUB: Self-training]

§12.3 Co-training and label propagation

[STUB: Co-training and label propagation]

§12.4 Consistency regularisation

[STUB: Consistency regularisation]

§12.5 When unlabelled data hurts

[STUB: When unlabelled data hurts]

§12.6 Contrastive learning in one page

[STUB: Contrastive learning in one page]

IV

Doing It Properly

Part IV: Contents

13	Evaluation and Model Selection	63
13.1	The confusion matrix	63
13.2	ROC and PR curves	63
13.3	Regression and probabilistic metrics	63
13.4	Cross-validation	63
13.5	Leakage	63
13.6	AIC, BIC and hyper-parameter search	63
13.7	Calibration	63
14	Statistics Interviewers Ask	65
14.1	Distributions	65
14.2	Estimators and confidence intervals	65
14.3	Hypothesis tests	65
14.4	Chi-square and ANOVA	65
14.5	Multiple testing	65
14.6	Correlation, causation and Simpson	65
15	Data Preparation and Feature Engineering	67
15.1	Scaling	67
15.2	Encoding categoricals	67
15.3	Missing values and outliers	67
15.4	Feature selection	67
15.5	Imbalance	67
15.6	Text features and TF-IDF	67
15.7	Time series	67
15.8	Recommenders	67
16	Optimisation for ML	69
16.1	Gradient descent on a quadratic	69
16.2	Momentum	69
16.3	Newton and quasi-Newton	69
16.4	Coordinate descent	69
16.5	Duality	69

13 Evaluation and Model Selection

§13.1 The confusion matrix

[STUB: The confusion matrix]

§13.2 ROC and PR curves

[STUB: ROC and PR curves]

§13.3 Regression and probabilistic metrics

[STUB: Regression and probabilistic metrics]

§13.4 Cross-validation

[STUB: Cross-validation]

§13.5 Leakage

[STUB: Leakage]

§13.6 AIC, BIC and hyper-parameter search

[STUB: AIC, BIC and hyper-parameter search]

§13.7 Calibration

[STUB: Calibration]

14 Statistics Interviewers Ask

§14.1 Distributions

[STUB: Distributions]

§14.2 Estimators and confidence intervals

[STUB: Estimators and confidence intervals]

§14.3 Hypothesis tests

[STUB: Hypothesis tests]

§14.4 Chi-square and ANOVA

[STUB: Chi-square and ANOVA]

§14.5 Multiple testing

[STUB: Multiple testing]

§14.6 Correlation, causation and Simpson

[STUB: Correlation, causation and Simpson]

15 Data Preparation and Feature Engineering

§15.1 Scaling

[STUB: Scaling]

§15.2 Encoding categoricals

[STUB: Encoding categoricals]

§15.3 Missing values and outliers

[STUB: Missing values and outliers]

§15.4 Feature selection

[STUB: Feature selection]

§15.5 Imbalance

[STUB: Imbalance]

§15.6 Text features and TF-IDF

[STUB: Text features and TF-IDF]

§15.7 Time series

[STUB: Time series]

§15.8 Recommenders

[STUB: Recommenders]

16 Optimisation for ML

§16.1 Gradient descent on a quadratic

[STUB: Gradient descent on a quadratic]

§16.2 Momentum

[STUB: Momentum]

§16.3 Newton and quasi-Newton

[STUB: Newton and quasi-Newton]

§16.4 Coordinate descent

[STUB: Coordinate descent]

§16.5 Duality

[STUB: Duality]



Interview Mode

Part V: Contents

17	Implement It	73
17.1	Linear models	73
17.2	Neighbours and Bayes	73
17.3	Trees and forests	73
17.4	Boosting	73
17.5	Clustering and PCA	73
17.6	SVM	73
18	Rapid-fire	75
18.1	Rapid-fire answers	75
19	Problems	77
19.1	Problems	77

17 Implement It

§17.1 Linear models

[STUB: Linear models]

§17.2 Neighbours and Bayes

[STUB: Neighbours and Bayes]

§17.3 Trees and forests

[STUB: Trees and forests]

§17.4 Boosting

[STUB: Boosting]

§17.5 Clustering and PCA

[STUB: Clustering and PCA]

§17.6 SVM

[STUB: SVM]

18 Rapid-fire

§18.1 Rapid-fire answers

[STUB: Rapid-fire answers]

19 Problems

§19.1 Problems

[STUB: Problems]

VI

Backmatter

A Hints to problems

- 1A.** Ask what plays the role of y at training time, and where it came from.
- 1B.** Residuals are $(1, 0, -2, 1)$.
- 1C.** Look at both the level and the gap.
- 1D.** Compare $\pi \phi(x - 2)$ with $(1 - \pi) \phi(x)$ and take logs.
- 1E.** Are the rows independent?
- 1F.** Each accuracy is approximately $\mathcal{N}(0.5, 0.25/n)$.
- 2A.** Square, and ignore the sign for strength.
- 2B.** Use the closure rules and the second-derivative test; for (f) look at the Hessian.
- 2C.** Set $\mathbf{A}\mathbf{w} - \mathbf{b} = \mathbf{0}$.
- 2D.** Use the first posterior as the new prior, or multiply likelihoods.
- 2E.** $\text{MAP} = (h + a - 1)/(n + a + b - 2)$.
- 2F.** Three cases: $a < 0$, $0 \leq a \leq 1$, $a > 1$.
- 3A.** $\bar{x} = 3$, $\bar{y} = 3$; compute $\sum (x - \bar{x})(y - \bar{y})$.
- 3B.** $b_1 = r s_y / s_x$.
- 3C.** Rank of $\mathbf{X}$; minimum-norm solutions.
- 3D.** Plug into $1 - (1 - R^2)(n - 1)/(n - p - 1)$.
- 3E.** $\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$ and $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}$.
- 3F.** Write the error $\mathbf{w}_t - \hat{\mathbf{w}}$ and show it is multiplied by $\mathbf{I} - \eta \mathbf{H}_J$ each step.

B Solutions to problems

- 1A.** (a) supervised regression. (b) unsupervised clustering/topic modelling. (c) self-supervised: the target frame is part of the input stream. (d) reinforcement learning: only delayed reward. (e) supervised; strictly ordinal regression, but usually treated as five-class classification — in an interview say that the order matters and that a model ignoring it penalises predicting 4 for a 5 as much as predicting 1.
- 1B.** $\text{MSE} = (1 + 0 + 4 + 1)/4 = 1.5$; $\text{MAE} = (1 + 0 + 2 + 1)/4 = 1$. With target 17 the last residual is -9 : MSE becomes $(1 + 0 + 4 + 81)/4 = 21.5$, MAE becomes $(1 + 0 + 2 + 9)/4 = 3$. Squared loss is far more sensitive to outliers, which is why MAE or Huber loss is used when targets have heavy tails.
- 1C.** A overfits (gap 0.15). C underfits (both low, gap 0.01). B has the best validation accuracy and a small gap; ship B (after confirming on the untouched test set). Note that A has the best training accuracy, and that the overfitting A still beats the underfitting C on validation (0.84 versus 0.70): overfitting is not automatically worse than underfitting; compare validation scores.
- 1D.** Predict 1 iff $\pi e^{-(x-2)^2/2} > (1 - \pi)e^{-x^2/2}$, i.e. $\ln \pi - (x - 2)^2/2 > \ln(1 - \pi) - x^2/2$, i.e. $2x - 2 > \ln \frac{1-\pi}{\pi}$, which is the claim. As $\pi \rightarrow 0$ the threshold goes to $+\infty$: when positives are rare you need overwhelming evidence to predict one, which is why accuracy-optimal classifiers on imbalanced data predict “negative” almost always (Chapter 13).
- 1E.** Images of the same patient land in both train and test; the network can recognise the patient (anatomy, implants, scanner artefacts) and copy that patient’s label. The test score measures memorisation, not diagnosis. Split by patient (`GroupKFold` with patient ID as the group), and ideally also by hospital/scanner if deployment will see new sites.
- 1F.** Standard deviation $= \sqrt{0.25/100} = 0.05$, so the best is about $0.5 + 2.25 \times 0.05 = 0.612$, matching the simulation 0.612 in the chapter. The optimism shrinks like $1/\sqrt{n}$ and grows like $\sqrt{2 \ln m}$: bigger validation sets help, comparing more models hurts.
- 2A.** 0.09 (9%) and 0.36 (36%). The second is stronger; the first is weaker and negative. Strength is $|r|$, direction is the sign, and r^2 is the explained fraction of variance in simple linear regression.
- 2B.** (a) convex: max of convex functions. (b) not convex: it equals x^2 on $[-1, 1]$ and 1 outside. The chord from $x = 0$ to $x = 2$ has midpoint value $(0 + 1)/2 = 0.5$, while $f(1) = 1 > 0.5$: the graph rises above the chord. (c) convex: logistic loss composed with an affine map, plus a norm. (d) not convex ($f'' = 6x < 0$ for $x < 0$). (e) convex ($f'' \geq 0$). (f) not convex: Hessian $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ has eigenvalues ± 1 , a saddle.
- 2C.** $\nabla f = \mathbf{A}\mathbf{w} - \mathbf{b}$, so $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}$ and $f(\mathbf{w}^*) = c - \frac{1}{2}\mathbf{b}^\top \mathbf{A}^{-1}\mathbf{b}$. If $\mathbf{A}$ is singular the minimiser is not unique (any null-space direction can be added) and may not exist if $\mathbf{b}$ has a component outside the range of $\mathbf{A}$ (f is then unbounded below). Ridge’s $+\lambda \mathbf{I}$ removes the problem.

- 2D.** Likelihood ratio of one positive is $0.95/0.10 = 9.5$. Prior odds $1/99$; after two positives the odds are $9.5^2/99 = 0.9116$, so $\mathbb{P}(D \mid ++)=0.477$. Updating in odds form is the fastest way to do this in an interview.
- 2E.** $\text{MLE} = 2/3 = 0.667$; $\text{MAP} = (2+1)/(3+20) = 3/23 = 0.130$. The MLE would put a 3-impression ad at the top of the ranking; the prior encodes that typical CTRs are low and shrinks small-sample estimates towards it. Use the shrunk estimate (or a lower confidence bound) for ranking.
- 2F.** The answer is the clip of a to $[0, 1]$. For $0 \leq a \leq 1$ both constraints are slack and both multipliers are 0. For $a > 1$ the constraint $x \leq 1$ binds, $x^* = 1$, and stationarity $2(1-a) + \mu_1 = 0$ gives $\mu_1 = 2(a-1) > 0$. For $a < 0$, $x^* = 0$ and the multiplier on $-x \leq 0$ is $\mu_0 = -2a > 0$. This box-constrained step is exactly the clipping in the SMO update for SVMs.
- 3A.** Centred products: $(-2)(-2) + (-1)(0) + 0 + (1)(2) + (2)(1) = 8$; $\sum(x - \bar{x})^2 = 10$. So $b_1 = 0.8$, $b_0 = 3 - 2.4 = 0.6$. Fitted: 1.4, 2.2, 3.0, 3.8, 4.6; residuals $-0.4, 0.8, -1.0, 1.2, -0.6$, $\text{RSS} = 0.16 + 0.64 + 1 + 1.44 + 0.36 = 3.6$, $\text{TSS} = 4 + 0 + 1 + 4 + 1 = 10$, $R^2 = 0.64$. At $x = 7$: 6.2.
- 3B.** $b_1 = 0.6 \cdot 10/2 = 3$, $b_0 = 60 - 3 \cdot 5 = 45$; prediction $45 + 24 = 69$. The reverse regression has slope $0.6 \cdot 2/10 = 0.12$, not $1/3$; the product of the slopes is $r^2 = 0.36$.
- 3C.** (a) $\mathbf{X}^\top \mathbf{X}$ is singular; no unique solution (any $w_1 + w_2 = c$ works). (b) It uses an SVD least-squares solver and returns the minimum-norm solution, splitting the weight equally. (c) Predictions are unchanged and well defined; only the coefficients are not identified. (d) Ridge has a unique solution and also splits the weight equally, $w_1 = w_2$.
- 3D.** A: $1 - 0.30 \cdot 59/56 = 0.6839$. B: $1 - 0.26 \cdot 59/49 = 0.6869$. B is marginally higher; the extra features carry a little signal. In practice, compare on validation data.
- 3E.** The first row of $\mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$ is the ones column: residuals sum to zero. $\hat{\mathbf{y}}^\top(\mathbf{y} - \hat{\mathbf{y}}) = \hat{\mathbf{w}}^\top \mathbf{X}^\top(\mathbf{y} - \hat{\mathbf{y}}) = 0$. Write $y_i - \bar{y} = (y_i - \hat{y}_i) + (\hat{y}_i - \bar{y})$, square and sum; the cross term is $\sum e_i \hat{y}_i - \bar{y} \sum e_i = 0$. Without an intercept this decomposition (and $R^2 \geq 0$) can fail.
- 3F.** With $\mathbf{H}_J = \frac{2}{n} \mathbf{X}^\top \mathbf{X}$, $\nabla J(\mathbf{w}) = \mathbf{H}_J(\mathbf{w} - \hat{\mathbf{w}})$, so $\mathbf{w}_{t+1} - \hat{\mathbf{w}} = (\mathbf{I} - \eta \mathbf{H}_J)(\mathbf{w}_t - \hat{\mathbf{w}})$. In the eigenbasis each component is multiplied by $1 - \eta \lambda_i$ per step; all shrink iff $|1 - \eta \lambda_i| < 1$ for every i , i.e. $0 < \eta < 2/\lambda_i$ for all i , i.e. $\eta < 2/\lambda_{\max}$. The slowest direction contracts by $1 - \eta \lambda_{\min}$, which is why the condition number $\lambda_{\max}/\lambda_{\min}$ governs speed.